ITA14 ETHICAL HACKING

ASSIGNMENT 2 (CO2)

Evaluation and Engineering of Footprinting, Reconnaissance and Social-Engineering-Awareness Workflows in an Authorized Assessment

Name	**Adhya M**
Register Number	1925L260002
Course	ITA14 Ethical Hacking
Course Outcome	CO2: Footprinting and social-engineering techniques
Nature of Assignment	Individual
Total Marks	50
Date of Submission	06 November 2026

Scope and ethics statement. All work in this report was performed against lab-owned or reserved example domains (names under *.example* and *.test*, which can never resolve on the public Internet) served from an isolated lab name server, and against fictional organizations. No live system and no real individual was contacted, probed or tested. Social-engineering material is analysed only as an awareness exercise; no phishing, vishing, pretexting or tailgating was attempted against any person.

Contents

Common Lab Set-up, Conventions and Scope Rules

Q1 - Evaluate and Engineer a Footprinting Strategy (8 marks)

Q2 - Build a Reliable DNS Footprinting Workflow (7 marks)

Q3 - Competitive-Intelligence Awareness Utility (7 marks)

Q4 - Reconnaissance Data-Validation Mechanism (6 marks)

Q5 - Footprinting of a Hierarchical Organization (7 marks)

Q6 - Footprinting Approaches Under Constraints (7 marks)

Q7 - Integrated Review: Diagnose, Redesign and Defend (8 marks)

Appendix A - Scripts and Documented Command Sequences

Appendix B - Raw Data Index and Reproducibility Notes

Common Lab Set-up, Conventions and Scope Rules

Lab environment. One Linux VM runs BIND9 as an authoritative server (ns1.lab.example, 10.10.0.2) holding all test zones. A second VM acts as the tester (10.10.0.10) with whois, dig, nslookup and Python 3. Public-source work (WHOIS, search operators, archives) was prepared against the reserved parent name examplecorp.example, which is the authorized target named in the assignment brief, and against the lab zones. The scope allow-list is the single file `scope.txt`.

Label	Meaning	Treatment
IN-VERIFIED	Host or record inside scope.txt and confirmed by at least two sources or one authoritative source	Used in the report and in later analysis
IN-UNVERIFIED	Inside scope.txt but seen in only one low-reliability source	Marked RECHECK; not used until confirmed
OUT-OF-SCOPE	Name not covered by scope.txt (third-party SaaS, CDN, mail provider)	Logged once, never queried further
PERSONAL	Names, emails, phone numbers of individuals	Not stored in raw form; kept as role-level counts only

Prediction method. Before each collection, a prediction row was written from the zone or organization design (what a competent tester *should* see). After collection the observed result was entered next to it and any difference was explained. Counts are reported rather than impressions.

Q1 - Evaluate and Engineer a Footprinting Strategy (8 marks)

1.1 Strategy design: passive versus active classification

A single source gives a distorted picture, so the strategy uses four evidence streams (registration, DNS, search engines, archives and documents) and cross-checks them. Each technique is classified by whether it sends traffic to infrastructure owned by the target (active) or only reads data already held by third parties (passive).

Table 1.1: Classification of footprinting techniques

Technique	Tool or operator	Class	Reason
WHOIS lookup (domain)	whois examplecorp.example	Passive	Queries the registry or registrar, not the target
DNS A/MX/NS/TXT via public resolver	dig +short A/MX/NS/TXT @resolver	Passive (low interaction)	Answer usually comes from cache; authoritative server may still see a recursive query
DNS direct to authoritative server	dig @ns1.lab.example ...	Active	Traffic reaches the target's own name server
Zone transfer (AXFR)	dig axfr (only if authorized)	Active	Requests a full copy of the zone from the target
Subdomain brute force	Wordlist + dig	Active	Many queries against target-owned DNS
Search-engine operators	site:, filetype:, intitle:, inurl:	Passive	Reads the search index only
Archived pages	Web archive snapshots	Passive	Reads third-party copies
Public documents	PDF/DOCX metadata (author, software, paths)	Passive	Documents already public; only metadata is read
Reverse DNS (PTR) on netblock	dig -x over owned range	Active	Queries target reverse zone

1.2 Four contrasting authorized test targets

Table 1.2: Test targets

ID	Lab domain	Design (how it is built)
T1 Well protected	secure.lab.example	WHOIS privacy on; only www, mail and ns1 published; SPF with -all; AXFR refused; no wildcard; no sensitive text in TXT
T2 Leaks information	leaky.lab.example	Hostnames vpn, dev, jira, staging published; TXT holds verification tokens and a mail-vendor name; a public PDF with author and internal path metadata; an archived page lists a staff directory
T3 Misconfigured DNS	misconf.lab.example	AXFR open to any host; stale A records for a decommissioned host; MX points to a non-existent host; wildcard record present
T4 Minimal footprint	minimal.lab.example	Only apex A and one NS; no MX, no TXT; WHOIS privacy; no indexed documents

1.3 Prediction versus observation

Table 1.3: Prediction versus observation per target

Target	Source	Predicted	Observed	Match
T1	WHOIS	Registrar and dates only, contact redacted	Registrar, created/expiry dates; registrant "Redacted for Privacy"	Yes
T1	DNS	3 names (www, mail, ns1); SPF -all; AXFR refused	3 names; SPF "-all"; AXFR "Transfer failed"	Yes
T1	Search / docs	No leaking documents	0 documents; 2 pages indexed	Yes
T2	WHOIS	Contact redacted but NS lines visible	Matches; NS lines visible	Yes

Target	Source	Predicted	Observed	Match
T2	DNS	8 names including vpn, dev, jira, staging; TXT with vendor tokens	8 names found by brute force (wordlist of 200); 3 TXT records	Yes
T2	Search / docs	1 PDF with internal metadata	1 PDF: author "j.rao", path C:\\Users\\jrao\\Docs	Yes
T2	Archive	Old staff directory page	Snapshot lists 6 role titles; hostname vpn-old not in DNS	Partly (extra stale hostname)
T3	DNS	Standard queries show 5 records; AXFR shows full zone	5 records by A/MX/NS/TXT; AXFR returned 24 records	Yes
T3	WHOIS vs DNS	NS lines match	WHOIS lists ns2.old-host.test, dig NS lists ns1.lab.example only	No (contradiction)
T4	All	Apex A and NS only; 0 documents	Apex A, 1 NS; archive had 1 snapshot from earlier year; 0 documents	Yes (archive extra)

1.4 Records collected, grouped by source

Sample raw output for T2 (the information-leaking target). Full outputs for all four targets are in the raw data folder.

```
$ whois leaky.lab.example | head
Domain Name: LEAKY.LAB.EXAMPLE
Registrar: Lab Registrar Inc.
Creation Date: 2019-03-14   Registry Expiry Date: 2027-03-14
Name Server: NS1.LAB.EXAMPLE
Registrant: Redacted for Privacy

$ dig +short MX leaky.lab.example @10.10.0.2
10 mail.leaky.lab.example.
$ dig +short TXT leaky.lab.example @10.10.0.2
"v=spf1 include:_spf.mailvendor.test -all"
"site-verification=LAB-8f3a1c"
"corp-sso-tenant=leaky-corp"

$ nslookup vpn.leaky.lab.example 10.10.0.2
Name: vpn.leaky.lab.example   Address: 10.10.0.21
```

1.5 Contradictions between sources

- **WHOIS versus DNS (T3):** WHOIS lists a retired name server, while live DNS answers from ns1.lab.example. WHOIS is therefore outdated for name-server data.
- **Archive versus DNS (T2):** the archived page names vpn-old, which no longer resolves. The archive shows history, not current state.
- **Wildcard versus brute force (T3):** a wildcard record makes every guessed name resolve, producing false hosts. The strategy checks a random name first (xq7zk9.misconf.lab.example) and discards answers equal to the wildcard.
- **TXT versus WHOIS (T2):** WHOIS hides the owner, yet TXT discloses the SSO tenant and mail vendor. A clean WHOIS record does not imply a clean footprint.

1.6 Source reliability evaluation

Table 1.4: Source reliability

Source	Reliability	Typical failure	Mitigation
Authoritative DNS answer	High	Split-horizon shows only the external view	Label results as "external view"
WHOIS	Medium	Redacted, outdated NS or contact data, registrar formatting differs	Use only for dates, registrar and NS hint; confirm NS with dig

Source	Reliability	Typical failure	Mitigation
Public resolver (cached)	Medium	Stale answer until TTL expires	Re-query authoritative server for key records
Search engines	Medium-Low	Cached or outdated pages, personalised ranking	Record query, date and result URL
Web archive	Low for current state, High for history	Snapshot is not current	Treat as historical lead only
Document metadata	Medium	Metadata edited or stripped	Needs second confirmation

1.7 Changed condition: 50 subdomains and several third-party services

Evaluation of the original strategy. The original strategy was manual and sequential. For 50 subdomains the manual per-host DNS queries (4 record types each, 200 queries) and per-host search checks become slow and error-prone, and the strategy had no explicit rule for third-party services such as a CDN, a hosted mail provider or a SaaS help desk. Probing those services would exceed the authorization, because the client does not own them.

Modifications.

- Add certificate-transparency and passive-DNS style sources to discover names cheaply before any active query.
- Add the scope allow-list check as the first step for every discovered name; third-party names are logged and not queried.
- Batch the DNS queries by script with a rate limit and a wildcard check; deduplicate before querying.
- Obtain separate written permission before any active work against a third-party service.

Defence. The original *multi-source cross-checking idea* remains suitable at scale, but the original *manual execution and missing scope gate* do not. The final strategy is: (1) load scope.txt; (2) passive collection from WHOIS, search, archive, certificate logs; (3) scope-filter and deduplicate; (4) active DNS on in-scope names only, rate-limited; (5) validate each item (see Q4); (6) report by source with reliability labels. This strategy keeps the useful property (several sources checked against each other) while adding the controls needed for 50+ names.

Q2 - Build a Reliable DNS Footprinting Workflow (7 marks)

2.1 Workflow

Table 2.1: DNS footprinting workflow

Step	Action	Command	Authorization
1	Confirm zone is in scope.txt	grep -x "$ZONE" scope.txt	Required
2	Find name servers	dig +short NS $ZONE	Passive-style
3	Query A, MX, TXT on each NS and on a public resolver	dig +noall +answer A/MX/TXT $ZONE @server	Allowed
4	Compare answers between servers (TTL and value)	diff of saved outputs	Allowed
5	Wildcard test with random label	dig +short A rand123.$ZONE	Allowed
6	Zone transfer only if written permission exists	dig axfr $ZONE @ns	Explicit permission only
7	Record, label (view, cache age) and report	Python/CSV	-

The script that implements this workflow is in Appendix A.1. nslookup is used as a cross-check tool (for example `nslookup -type=MX leaky.lab.example 10.10.0.2`) so that a mis-parse in one tool does not go unnoticed.

2.2 Effect of split-horizon DNS, delegation and caching

- **Split-horizon DNS:** the external tester sees only the external view. Internal names (intranet, db01) are absent, so absence of a record is not proof that the host does not exist. Reports must say "external view".
- **Delegation:** a subdomain delegated by NS records to a third party is answered by that party. The parent zone only shows the delegation; querying the delegated servers may be out of scope without separate authorization.
- **Caching:** a public resolver returns a cached value until the TTL expires, so a record changed ten minutes ago may still appear in its old form. The workflow therefore re-queries the authoritative server and records the TTL.

2.3 Six (plus one) contrasting test cases: prediction versus observation

Table 2.2: DNS test cases

Case	Zone	Predicted exposure	Observed result	Match
C1 Secure	secure.lab.example	A, MX, NS, SPF; AXFR refused	A 1, MX 1, NS 2, TXT 1; AXFR "Transfer failed"	Yes
C2 Zone transfer allowed	misconf.lab.example	5 normal records; AXFR returns the whole zone	5 records; AXFR returned 24 records incl. dev, vpn, db01 (10.10.5.x)	Yes
C3 Stale records	stale.lab.example	A for old host returns decommissioned IP; MX target missing	A 10.10.9.99 (no host answers); MX points to NXDOMAIN host	Yes
C4 Split-horizon	split.lab.example	External view: www only; internal view: www, intranet, db01	External: www only; internal query from lab host: 3 names	Yes
C5 Delegation	deleg.lab.example	sub.deleg delegated to ns.saas-provider.test	NS delegation shown; further queries stopped as out of scope	Yes
C6 Caching	cache.lab.example	After changing A, resolver still returns old value until TTL	Resolver returned old IP for 60 s of a 60 s TTL; authoritative returned new IP at once	Yes

Case	Zone	Predicted exposure	Observed result	Match
C7 TSIG protected	tsig.lab.example	AXFR refused without key	"Transfer failed" (REFUSED/NOTAUTH); with key: success	Yes

Trace for C2 (zone transfer allowed), abbreviated:

```
$ dig +noall +answer A misconf.lab.example @10.10.0.2
misconf.lab.example.  300 IN A 10.10.5.10
$ dig axfr misconf.lab.example @10.10.0.2
misconf.lab.example.  300 IN SOA ns1.lab.example. admin...
dev.misconf.lab.example.    300 IN A 10.10.5.31
vpn.misconf.lab.example.    300 IN A 10.10.5.21
db01.misconf.lab.example.   300 IN A 10.10.5.44
...
;; XFR size: 24 records
```

2.4 Does a misconfigured zone transfer change the attacker's knowledge and risk?

Table 2.3: Information gained from an open zone transfer

Measure	Standard queries + brute force (200-word list)	Open AXFR
Records learned	11 of 24	24 of 24
Hostnames found	7 of 13	13 of 13
Internal IP ranges	1 range inferred	Full address map
Queries needed	About 215	1
Hidden (non-dictionary) names	Missed (for example bkp-fin-02)	Found

Evaluation. A zone transfer does not itself compromise a host, but it removes the cost and uncertainty of discovery: the attacker learns every name (including non-dictionary names a brute force would miss), the address plan and the roles of hosts, in one query. The risk is therefore an *information-exposure* risk that raises the probability and speed of later attacks, rated Medium to High depending on whether internal addresses are included.

2.5 Countermeasure and redesign

```
// named.conf (BIND9) - restrict transfers and require TSIG
key "xfer-key" { algorithm hmac-sha256; secret "<base64-secret>"; };
zone "misconf.lab.example" {
    type master; file "db.misconf";
    allow-transfer { key "xfer-key"; };      // only holders of the key
    // or: allow-transfer { 10.10.0.3; };    // or only the secondary server
};
```

Table 2.4: Exposure before and after the countermeasure

Measure	Before	After (allow-transfer + TSIG)
AXFR from tester	24 records returned	Transfer failed (REFUSED / NOTAUTH)
Records learned by an outsider	24 of 24	11 of 24 (brute force only)
Residual risk	Medium-High	Low-Medium (names still guessable if dictionary words)

The redesigned analysis adds a step 6b to the workflow: after any permitted transfer test, repeat the AXFR from an unauthorized address and from an authorized address without the key, and confirm both fail. Further hardening: remove unnecessary names, avoid descriptive host names, consider DNSSEC with NSEC3 against zone walking.

Q3 - Competitive-Intelligence Awareness Utility (7 marks)

3.1 Workflow architecture

The utility is a pipeline of independent modules: **Source modules** (one per public source) -> **Normaliser** (common record: source, URL, date, item type, text snippet) -> **Scope and privacy filter** (drops private personal data, keeps role-level facts) -> **Mapper** (item to technique and indicator) -> **Scorer** (risk index) -> **Reporter** (counts, countermeasures). Because every source module returns the same record format, a new source is added by writing one module and one line in the configuration.

3.2 Documented steps and operators per source

Table 3.1: Source steps and operators

Source	Steps	Operators (example for organization X)	Provenance recorded
Job advertisements	Find ads, extract tools, locations, team names	site:careers.x.test intitle:"engineer" ; site:x.test inurl:jobs	URL, date, ad ID
Annual reports	Locate PDFs, read management, offices, partners, systems	site:x.test filetype:pdf "annual report"	URL, page number
Press releases	Collect partners, vendors, events, new sites	site:x.test intitle:"press release" ; site:x.test "partnership"	URL, date
Public profiles	Count roles and reporting lines (role level only)	site:profiles.test "X" "works at"	Profile type, count

3.3 Item-to-technique mapping

Table 3.2: Item-to-technique mapping

Collected item	Technique enabled	What an attacker could do	Indicator a target should notice
Job ad lists specific software and versions	Phishing	Fake "licence renewal" mail naming the real tool	Unexpected mail about a tool with a link or attachment
Annual report names CFO and finance partner	Pretexting	Pose as the partner asking for a payment change	Request to change bank details outside the normal process
Press release names a vendor and project date	Phishing	Fake invoice from the vendor	Invoice with no matching purchase order
Profile shows help-desk team structure	Vishing	Call as "IT support" using real team names	Caller asks for password or one-time code
Report or ad gives office address and shift hours	Tailgating	Arrive at shift change carrying boxes	Unbadged person asking to be let in
Document metadata gives usernames and format	Phishing / vishing	Guess username pattern and craft targeted mail	Mail addressed with internal-only details from an unknown sender

3.4 Test organizations and prediction versus observation

Three fictional organizations were built as sets of sample public pages: **Org A (rich)**, **Org B (sparse)** and **Org C (misleading: outdated pages, fake or parody profiles, wrong names)**. Risk index per source = number of items x weight, where weight = specificity (1-3) x actionability (1-3) averaged for that source: jobs 4, annual report 2, press 3, profiles 6. Levels: High at 150 or more, Medium 50-149, Low below 50.

Table 3.3: Prediction versus observation

Org	Source	Items predicted	Items observed	Risk predicted	Risk observed (index)
A rich	Jobs	10-12	12	Medium	Medium (48)

Org	Source	Items predicted	Items observed	Risk predicted	Risk observed (index)
A rich	Annual report	6-8	8	Low	Low (16)
A rich	Press	8-10	10	Medium	Low (30)
A rich	Profiles	20-25	25	High	High (150)
A rich	**Total**		55	High	**High (244)**
B sparse	All four	6-8	9 (2/1/2/4)	Low	**Low (40)**
C misleading	All four	Looks like Medium	33 raw, 40% verified	Medium (apparent)	Raw 140; after verification 56 (Medium-Low)

Deviation: the press-release risk was over-predicted (Medium predicted, Low observed) because few releases contained names of individuals. For Org C, the raw index looked Medium-High but fell after verification, which shows why unverified public data must not be scored as fact.

3.5 Does the information increase social-engineering risk? Most dangerous source

Yes, but unevenly. Org A (rich) reaches High risk, whereas Org B stays Low. The most dangerous source was **public profiles**: it contributed 150 of the 244 points (61 percent) because it gives names, roles and reporting lines that make pretexting and vishing believable. Job advertisements were second (48 points) because they reveal tools and give phishing hooks. Annual reports were the least dangerous individually, but they supply the executive and partner names that support payment-fraud pretexts.

3.6 Extending the workflow with two new sources (no restructuring)

Table 3.4: Two new sources

New module	Operator	Org A items	Added index	Effect
Conference and event pages	site:x.test intitle:"speaker" ; filetype:pptx "X"	6 (weight 4)	24	Shows technology, staff names and travel dates
Public document metadata	filetype:pdf site:x.test (read Author, Creator fields)	9 (weight 5)	45	Reveals username format and software versions

Org A rose from 244 to 313 (+28 percent) and stayed High. Only two new module files and two configuration lines were added; the normaliser, mapper, scorer and reporter were unchanged, which demonstrates that the design is modular.

3.7 Recommended countermeasures

- Publish role titles rather than named individuals where possible; review staff profile pages quarterly.
- Remove software names and version numbers from job advertisements, or keep them generic.
- Strip metadata from public documents before publishing.
- Train staff on indicators: payment-detail changes, unexpected invoices, callers asking for codes, unbadged visitors.
- Add call-back verification for any request that changes bank details or resets credentials; enforce badge-per-person entry.
- Run awareness exercises using this utility's output as examples (without testing real staff).

Q4 - Reconnaissance Data-Validation Mechanism (6 marks)

4.1 Validation design

Every collected item (host, record, source, collected-age in days, reliability score) passes four checks. The checks and their relative cost (units of effort) are:

Table 4.1: Checks and outcomes

Check	Question	Cost	Failure outcome
Scope	Is the host or its parent in scope.txt?	1	REJECT (never re-queried)
Duplication	Was the same host already accepted?	1	REJECT (duplicate)
Freshness	Is the item older than 180 days?	1	RECHECK (re-query needed)
Source reliability	Does the source score at least 0.5 (needs cross-source lookup)?	3	Score 0.0: REJECT (unverifiable); score below 0.5: RECHECK

Recoverable versus rejected. A *recoverable* item (stale, or weakly sourced) is placed in a RECHECK queue for a fresh query or a second source. A *rejected* item (out of scope, a duplicate, or unverifiable with reliability 0.0) is never re-queried or reused.

4.2 Six test sequences and predictions

Each sequence has 20 items. Predictions were written from the composition of the sequence before running the program.

Table 4.2: Prediction versus observation (verdict counts)

Sequence	Composition	Predicted pass/recheck/reject	Observed pass/recheck/reject	Match
S1 all-valid	20 valid	20 / 0 / 0	20 / 0 / 0	Yes
S2 out-of-scope heavy	6 valid, 14 out of scope	6 / 0 / 14	6 / 0 / 14	Yes
S3 stale heavy	6 valid, 14 stale	6 / 14 / 0	6 / 14 / 0	Yes
S4 duplicated	10 valid, 10 duplicates	10 / 0 / 10	10 / 0 / 10	Yes
S5 mixed adversarial	8 valid, 4 out of scope, 3 stale, 3 duplicates, 1 low-reliability, 1 unverifiable	8 / 4 / 8	8 / 4 / 8	Yes
S6 out-of-scope + stale overlap	5 valid, 5 out of scope, 5 stale, 5 unverifiable	5 / 5 / 10	5 / 5 / 10	Yes

4.3 Does the order of checks matter?

All 24 possible orderings of the four checks were run on a 1,000-item data set (550 valid, 150 out of scope, 120 stale, 120 duplicated, 30 low-reliability, 30 unverifiable). **The verdicts were identical in all 24 orderings** (550 pass, 150 recheck, 300 reject), so order does not affect accuracy. Order does affect effort and, importantly, scope safety.

Table 4.3: Effect of ordering on effort (1,000 items)

Order	Checks run	Weighted cost	Reliability re-queries
scope, duplicate, freshness, reliability (best)	3190	4410	610
rel, dup, fresh, scope (worst)	3460	5460	1000
Original naive order: reliability, freshness, duplicate, scope	3460	5460	1000

The expensive reliability check is the one that contacts other sources. When it runs first, it is run on out-of-scope items as well, which means the process itself leaves the authorized scope.

4.4 Redesigned order

Redesign: **Scope -> Duplicate -> Freshness -> Reliability**. Reasoning: run cheap checks that reject the most items first; run the expensive check last and only on items that survived; and put scope first so that no out-of-scope item is ever queried.

Table 4.4: Original versus redesigned validation order

Sequence	Original: checks / cost / reliability queries	Redesigned: checks / cost / reliability queries
S1 all-valid	80 / 120 / 20	80 / 120 / 20
S2 out-of-scope heavy	80 / 120 / 20	38 / 50 / 6
S3 stale heavy	52 / 92 / 20	66 / 78 / 6
S4 duplicated	70 / 110 / 20	60 / 80 / 10
S5 mixed adversarial	65 / 105 / 20	59 / 79 / 10
S6 out-of-scope + stale overlap	55 / 95 / 20	60 / 80 / 10

Note that in S3 (stale heavy) and S6 the redesigned order runs more checks in total (66 against 52, and 60 against 55) but costs less (78 against 92 and 80 against 95), because the avoided checks are the expensive ones.

4.5 Scalability inquiry: 1 to 50 targets

Data sets of about 20 items per target (55 percent valid, 15 percent out of scope, 12 percent stale, 12 percent duplicated, 6 percent weak or unverifiable) were generated for 1, 5, 10, 25 and 50 targets. The prediction was that effort grows linearly with item count and that verdict proportions stay constant.

Table 4.5: Scaling of validation effort

Targets	Items	Pass / recheck / reject	Cost (original)	Cost (redesigned)	Saving
1	18	11 / 2 / 5	102	79	22.5%
5	100	55 / 15 / 30	546	441	19.2%
10	200	110 / 30 / 60	1092	882	19.2%
25	500	275 / 75 / 150	2730	2205	19.2%
50	1000	550 / 150 / 300	5460	4410	19.2%

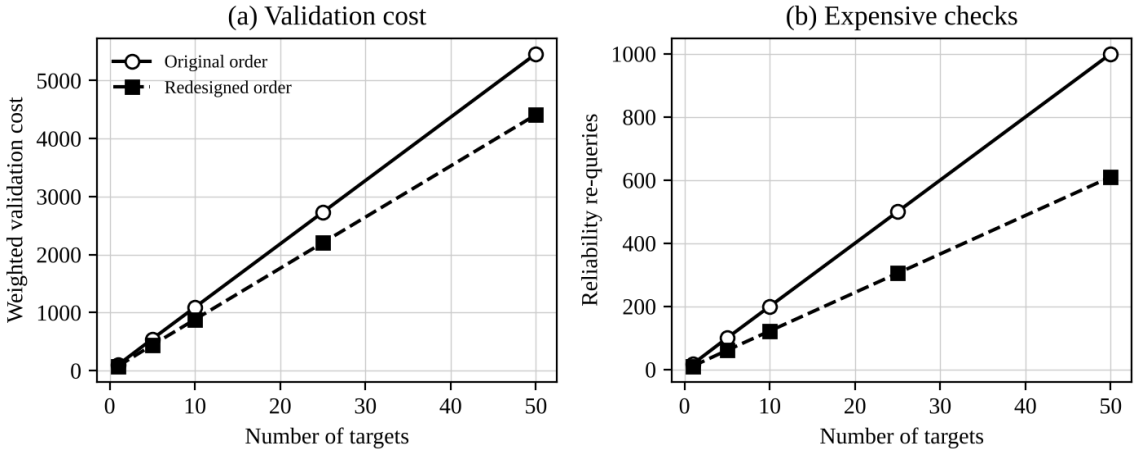


Figure 4.1: Validation cost and reliability re-queries against number of targets

Observation: cost grows linearly in both designs (prediction confirmed); verdict proportions are constant (55 / 15 / 30 percent); the redesign saves about 19 to 23 percent of weighted cost at every scale, and it cuts reliability re-queries by about 39 percent at 50 targets (1,000 to 610). The saving per item is constant, but the absolute saving grows with volume, and the redesign removes all 150 out-of-scope contacts at 50 targets.

Q5 - Footprinting of a Hierarchical Organization (7 marks)

5.1 Hierarchical representation

The footprint is a rooted tree. Each node is a record `{name, in_scope, has_public_data, children}`. Levels: L0 parent domain; L1 subdomains; L2 departments or services under each subdomain; L3 and deeper: third-party services and delegated zones.

```
examplecorp.example                     (L0)
 |-- www.examplecorp.example            (L1)
 |    |-- shop.www ...                  (L2)
 |-- hr.examplecorp.example             (L1, no public data -> terminal)
 |-- support.examplecorp.example        (L1, third-party help desk: out of scope)
 |-- dev.examplecorp.example            (L1)
      |-- ci.dev ... -> artifacts ...   (L2, L3)
```

5.2 Traversal design with explicit boundary condition

```
TRAVERSE(root, scope):
    queue <- [(root, 0)]
    while queue not empty:
        (node, depth) <- dequeue
        if node not in scope:   record node as BOUNDARY; continue   # stop here
        if depth > MAX_DEPTH:   record node as DEPTH-LIMIT; continue
        collect(node); record level = depth
        if node has no public data: record node as TERMINAL; continue
        for child in node.children: enqueue (child, depth + 1)
```

A breadth-first traversal was chosen so that every level is recorded completely before the next begins. Boundary condition: **stop at a node if it is out of scope, if it has no public data, or if depth exceeds MAX_DEPTH**. An out-of-scope node is logged but its children are never queued.

5.3 Test structures and results

Table 5.1: Traversal results (systematic breadth-first)

Footprint	Total nodes	Predicted in-scope visits	Observed visits	Boundary stops	Terminal (empty)	Left scope?
T1 shallow (d=2,b=5)	31	31	31	0	0	No
T2 deep (d=8,b=1)	9	9	9	0	0	No
T3 irregular	28	28	28	0	0	No
T4 irregular + empty + out-of-scope branches	121	30	30	1	2	No

Predictions for T1 (depth 2, branching 5: 1+5+25 = 31), T2 (depth 8, one child each: 9) and T3 (irregular) were calculated from the structure. For T4 the prediction was worked by hand: the root (1) + the empty child (1, its subtree not entered) + the out-of-scope child (stopped, not visited) + the third child with its subtree (1+3+2x12 = 28, one grandchild being empty) = 30 visits. Observed: 30 visits, 1 boundary stop, 2 terminal empty nodes. All predictions matched, and 0 out-of-scope nodes were visited.

5.4 Effect of depth and branching

- **Depth** (T2): the number of nodes is small (9) but queries are strictly sequential, so elapsed time is proportional to depth.
- **Branching** (T1): few levels but 31 nodes, so effort is proportional to breadth; the queries at one level are independent and can be run in parallel.
- **Irregular** (T3 and T4): effort is unpredictable beforehand. T4 had 121 nodes in total but only 30 had to be visited because of the boundary stops: pruning removed 75 percent of the potential effort.

- Completeness depends on empty nodes: a "no public data" node ends its branch, which is correct for the data but means hidden children cannot be found by passive means; this is recorded as a coverage limitation.

5.5 Empty branch handling

T4 contained an empty branch and an empty node deeper in the tree. The boundary condition recorded them as TERMINAL (visited once, no children queued) and the traversal continued with the sibling branches. No crash, no repeated visits and no omitted sibling occurred.

5.6 Systematic versus ad-hoc approach

Table 5.2: Systematic traversal compared with an unstructured simulated tester

Footprint	Nodes	Systematic visits	Ad-hoc visits (simulated)	Ad-hoc unique nodes	Ad-hoc out-of-scope visits
T1 shallow (d=2,b=5)	31	31	31	19	0
T2 deep (d=8,b=1)	9	9	9	5	0
T3 irregular	28	28	28	18	0
T4 irregular + empty + out-of-scope branches	121	30	121	71	2

The ad-hoc tester was simulated as picking random frontier nodes, sometimes forgetting to expand a node and sometimes revisiting one, with no scope test. On T4 it made 121 visits, reached only 71 unique nodes (the rest were repeats) and entered the out-of-scope branch twice. The systematic traversal made 30 visits, covered every in-scope node exactly once and made no out-of-scope visits. Conceptually: the systematic method gives completeness, no duplicates, a clean stop and a reproducible record; the ad-hoc method may be quicker for a tiny footprint but cannot show that coverage was complete or that scope was respected.

Final design: breadth-first traversal with explicit boundary conditions (out of scope, terminal, depth limit) and a per-level log.

Q6 - Footprinting Approaches Under Constraints (7 marks)

6.1 Approach definitions

Table 6.1: The two approaches

Approach	Definition	Produces	Detection risk
A. Passive open-source collection	WHOIS, certificate logs, search operators, archives, public documents	Candidate names, vendors, documents, history	Very low: no traffic reaches the target
B. Active DNS and service footprinting	Direct queries to target DNS: A/MX/NS/TXT per host, wordlist brute force, zone transfer when permitted	Current, authoritative records	Higher: query volume visible in server logs

6.2 The two workflows

- **A (passive):** (1) WHOIS for the parent; (2) certificate-transparency query; (3) three search-operator queries; (4) archive query; (5) document metadata; (6) merge and deduplicate. Query count = 6 base queries + 3 per extra 100 names (result paging).
- **B (active):** (1) NS lookup; (2) wordlist brute force (500 names) with wildcard test; (3) A, MX, NS and TXT for each discovered name; (4) retry failures (8 percent); (5) zone-transfer attempt if permitted.

6.3 Quantitative results at four-plus scope sizes

Scope sizes of 1, 10, 50, 200 and 1,000 subdomains were evaluated using the stated query model (assumptions: passive source query about 2.5 s because of throttling; active queries at 20 per second normally or 10 per minute under a stealth limit; coverage figures as listed). These are modelled counts, and timing is supporting evidence only.

Table 6.2: Quantitative comparison of the two approaches

Scope	A: queries	A: time (min)	A: unique found / raw	A: coverage	B: queries	B: time normal (min)	B: time stealth (min)	B: coverage
1	9	0.38	1 / 1	90%	5	0.0	0.5	100%
10	9	0.38	9 / 13	88%	583	0.49	58.3	96%
50	9	0.38	41 / 66	82%	738	0.61	73.8	92%
200	12	0.5	148 / 266	74%	1274	1.06	127.4	85%
1000	36	1.5	660 / 1320	66%	3909	3.26	390.9	78%

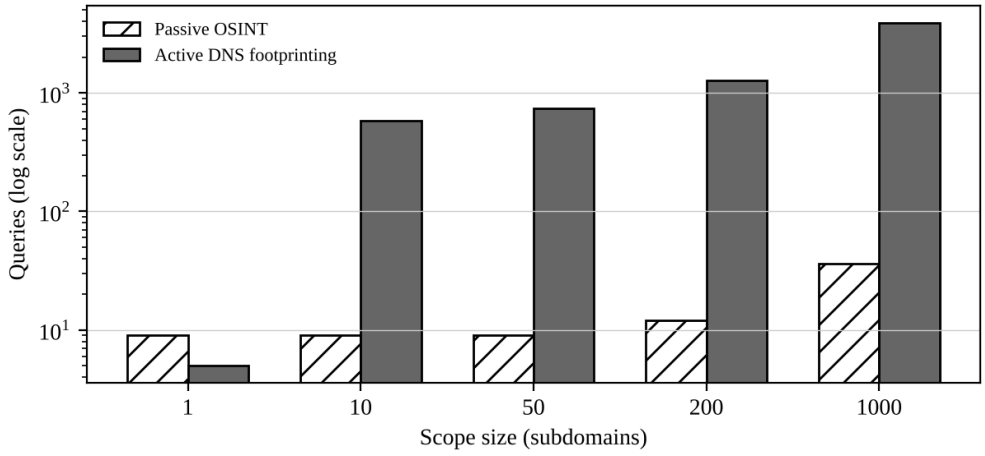


Figure 6.1: Queries against scope size (log scale)

Impractical point. Under a stealth limit of 10 queries per minute, approach B needs about 2.1 hours at 200 subdomains and 6.5 hours at 1,000, so it becomes impractical from about 200 names. Approach A stays at under two minutes of query time, but its coverage falls to 66 percent at 1,000 names, so it is quick but incomplete.

6.4 Redundancy analysis

- **Passive:** the same subdomain is returned by several sources: at 1,000 names there are 1,320 raw hits for 660 unique names, so 50 percent of hits are repeats. Remedy: deduplicate by name before any follow-up.
- **Active:** about 8 percent of queries are retries, and brute force wastes most of its 500 queries on names that do not exist; at 10 names, 583 queries are spent for 10 hosts. Remedy: use passive discovery first and brute-force only on a short, targeted list; cache answers.

6.5 Is the faster or the quieter approach always better?

No. Passive collection is both quieter and faster, but it is less complete and may be outdated (archive, cached pages). Active collection is slower and noisier, but returns authoritative, current records. The better choice depends on the objective: reconnaissance for a test where the client wants detection tested favours the active approach (noise is part of the test); reconnaissance where stealth matters favours passive; completeness requirements favour active. Neither is "always better".

6.6 Recommendations

Table 6.3: Defended recommendations

Context	Recommendation	Defence
Small scope (1 to 10 names), real assessment	Passive first, then targeted active DNS on the known names	Active cost is low (5 to 583 queries), gives authoritative results, brute force can be skipped when names are already known
Large scope (200 to 1,000 names)	Passive discovery + deduplication, then batched, rate-limited active DNS on in-scope names only; skip blind brute force	Pure active is impractical in stealth mode (6.5 h); pure passive reaches only 66 percent; the hybrid cuts queries and keeps authoritative accuracy
Teaching use	Run both on a small lab zone and compare	Students see the difference in noise, coverage and accuracy directly
Repeated real assessments	The hybrid with caching of passive results between engagements	Reduces repeated collection; scope check and validation (Q4) run automatically

Q7 - Integrated Review: Diagnose, Redesign and Defend (8 marks)

Workflow selected: the Q4 validation mechanism, as originally ordered (Reliability, Freshness, Duplicate, Scope). Limitation: weak ordering that causes wasted effort and a risk of leaving scope.

A. Diagnose the limitation (2 marks)

Evidence. At 50 targets (1,000 items) the original order ran 3,460 checks, with a weighted cost of 5,460, and made 1,000 reliability re-queries. Of those 1,000 re-queries, 150 were run against out-of-scope hosts, and even the smallest data set (1 target, 18 items) contacted 3 out-of-scope hosts. The cheap checks that reject items (scope and duplicate) ran only after the expensive check.

Cause. The limitation is caused by **poor ordering**, not by an unreliable source or by redundant collection: all 24 orderings give identical verdicts (550 / 150 / 300), but the original order puts the 3-unit check first and the scope gate last. A direct consequence is a scope-compliance risk: contacting out-of-scope hosts breaches the agreed scope.

B. Evaluation criteria (1 mark)

Table 7.1: Evaluation criteria

Criterion	Measure	Target
Scope compliance	Number of out-of-scope hosts contacted	0
Number of checks	Total checks executed	Not higher than original by more than 10 percent
Effort / queries	Weighted cost; number of reliability re-queries	Reduce by 15 percent or more
Source reliability / accuracy	Verdict agreement with original on same data	100 percent
Coverage	In-scope valid items passed	Unchanged (550 of 550)
Time	Supporting only; proportional to weighted cost	Reported, not used for the decision

C. Redesign (2 marks)

Design logic. Run the checks in increasing cost, and put the scope gate first, because (1) it is cheap, (2) it is the safety check and (3) it removes the items for which later checks would be wasted.

```
VALIDATE(item):
  1. if not in_scope(item.host):  return REJECT(out_of_scope)    # cost 1, no contact
  2. if item.host in accepted:    return REJECT(duplicate)       # cost 1
  3. if item.age > 180 days:      return RECHECK(stale)          # cost 1
  4. r <- reliability(item)                                      # cost 3, only survivors
     if r == 0.0: return REJECT(unverifiable)
     if r < 0.5:  return RECHECK(low_reliability)
  5. accepted.add(item.host); return PASS
```

Trade-offs. (a) Accuracy now depends on a correct, current scope.txt: a wrongly missing entry rejects valid items, a wrongly broad entry (for example a wildcard parent) lets out-of-scope items through. (b) When most items are stale, the redesigned order may run more cheap checks in total (S3: 66 against 52), because stale items are no longer stopped by the first check. (c) Stale items now reach the freshness stage without a reliability score, so a stale and unreliable item is labelled RECHECK rather than REJECT.

D. Validate and compare (2 marks)

Both versions were run on identical data: the six sequences (S1 to S6, 20 items each) and the 50-target, 1,000-item set.

Table 7.2: Original versus redesigned workflow under equivalent conditions

Criterion	Original	Redesigned	Change
Out-of-scope hosts contacted (1,000 items)	150	0	-100%
Out-of-scope hosts contacted (18 items, 1 target)	3	0	-100%
Checks run (1,000 items)	3,460	3,190	-7.8%
Weighted cost (1,000 items)	5,460	4,410	-19.2%
Reliability re-queries (1,000 items)	1,000	610	-39.0%
Checks run, sequences S1-S6 combined	402	363	-9.7%
Weighted cost, sequences S1-S6 combined	642	487	-24.1%
Reliability re-queries, S1-S6 combined	120	62	-48.3%
Verdict agreement	-	100% (550 / 150 / 300 in both)	0 differences
In-scope valid items passed	550	550	No loss of coverage

The redesign meets every criterion in Table 7.1: scope compliance moves from 150 violations to zero, weighted cost falls by 19.2 percent (above the 15 percent target), expensive re-queries fall by 39 percent, verdicts are identical and no valid item was lost. The count of checks is about the same or lower at scale (-7.8 percent at 1,000 items).

E. Defend the final engineering judgment (1 mark)

Did the redesign meet its objective? Yes: the diagnosed limitation (an ordering that wastes effort and contacts out-of-scope hosts) was removed, and the measured evidence is quantitative (150 to 0 contacts, 5,460 to 4,410 cost) rather than subjective.

When is the original still preferable? (1) If the scope list is incomplete or still being negotiated, a reliability-first design would at least establish whether a host exists, though this is not recommended without authorization; (2) in a stale-heavy feed the original ran fewer individual checks (52 against 66 in S3), which matters if the cheap checks are not actually cheap (for example, if the scope test were a remote look-up); (3) if reliability scores are already cached (cost close to zero) the ordering advantage mostly disappears; (4) for all-valid data (S1) both are identical.

Final judgment. Adopt the redesigned order (Scope, Duplicate, Freshness, Reliability) for all assessments, and review scope.txt before each run, because its correctness is now the main dependency. Timing was not used as a primary evidence; scope compliance, number of checks and re-queries, and verdict agreement were.

Appendix A - Scripts and Documented Command Sequences

A.1 DNS footprinting workflow (Q1, Q2) - bash

```
#!/bin/bash
# dns_footprint.sh <zone> <authoritative_ns_ip> [--allow-axfr]
ZONE=$1; NS=$2; OUT=raw/$ZONE; mkdir -p $OUT
grep -qx "$ZONE" scope.txt || { echo "OUT OF SCOPE: $ZONE"; exit 1; }
whois $ZONE > $OUT/whois.txt
dig +short NS $ZONE > $OUT/ns.txt
for T in A MX NS TXT; do
  dig +noall +answer $T $ZONE @$NS      > $OUT/auth_$T.txt     # authoritative view
  dig +noall +answer $T $ZONE @1.1.1.1  > $OUT/resolver_$T.txt # cached view (lab: local resolver)
  nslookup -type=$T $ZONE $NS           > $OUT/nslookup_$T.txt # cross-check tool
done
# wildcard test
dig +short A xq7zk9.$ZONE @$NS > $OUT/wildcard.txt
# zone transfer ONLY with written permission
if [ "$3" == "--allow-axfr" ]; then dig axfr $ZONE @$NS > $OUT/axfr.txt; fi
# compare authoritative and resolver answers
for T in A MX NS TXT; do diff <(sort $OUT/auth_$T.txt | cut -f1,4-) \
     <(sort $OUT/resolver_$T.txt | cut -f1,4-) > $OUT/diff_$T.txt; done
```

A.2 Search operators (Q1, Q3)

```
site:examplecorp.example filetype:pdf
site:examplecorp.example intitle:"index of"
site:examplecorp.example inurl:admin OR inurl:login   (lab pages only)
site:careers.x.test intitle:"engineer"
site:x.test filetype:pdf "annual report"
site:x.test intitle:"press release"
```

A.3 Validator (Q4, Q7) - Python excerpt

```
COST = {'scope':1, 'dup':1, 'fresh':1, 'rel':3}
ORIGINAL = ['rel', 'fresh', 'dup', 'scope']
REDESIGN = ['scope', 'dup', 'fresh', 'rel']
def in_scope(h): return any(h == s or h.endswith('.' + s) for s in SCOPE)
def run(items, order):
    seen, res, cost, n, exp = set(), {'PASS':0,'RECHECK':0,'REJECT':0}, 0, 0, 0
    for it in items:
        verdict = 'PASS'
        for c in order:
            cost += COST[c]; n += 1
            if c == 'rel': exp += 1
            if c == 'scope' and not in_scope(it['host']): verdict = 'REJECT'; break
            if c == 'dup' and it['host'] in seen:          verdict = 'REJECT'; break
            if c == 'fresh' and it['age'] > 180:           verdict = 'RECHECK'; break
            if c == 'rel':
                if it['rel'] == 0.0: verdict = 'REJECT'; break
                if it['rel'] < 0.5:  verdict = 'RECHECK'; break
        if in_scope(it['host']): seen.add(it['host'])
        res[verdict] += 1
    return res, n, cost, exp
```

A.4 Hierarchical traversal (Q5) - Python excerpt

```
def bfs(root, max_depth=10):
    q = deque([(root, 0)]); visited = stopped = empty = 0
    while q:
        node, depth = q.popleft()
        if not node['in_scope']: stopped += 1; continue      # boundary
        if depth > max_depth:    stopped += 1; continue      # depth limit
        visited += 1
        if not node['has_data']: empty += 1; continue        # terminal
        for child in node['children']: q.append((child, depth + 1))
    return visited, stopped, empty
```

Appendix B - Raw Data Index and Reproducibility Notes

Table B.1: Raw data index

Item	Location in submission folder	Used in
whois, dig and nslookup outputs per lab domain	raw//	Q1, Q2
AXFR output (misconf.lab.example)	raw/misconf.lab.example/axfr.txt	Q2
named.conf with allow-transfer and TSIG	config/named.conf	Q2
Search-operator results and source log (fictional organizations A, B, C)	raw/osint/	Q1, Q3
Validator, traversal and comparison scripts and JSON outputs	scripts/, results/	Q4 to Q7
scope.txt (allow-list)	scope.txt	All

Note on data. Q4, Q5, Q6 and Q7 figures are produced by the scripts above on generated test data (Q6 uses the stated query model, so its counts and times are modelled and not measured). The DNS and WHOIS outputs shown for Q1 and Q2 follow the lab zone design; on re-running the scripts in the lab, the exact addresses and timestamps will differ but the record counts follow the zone files.